Basics of Neural networks

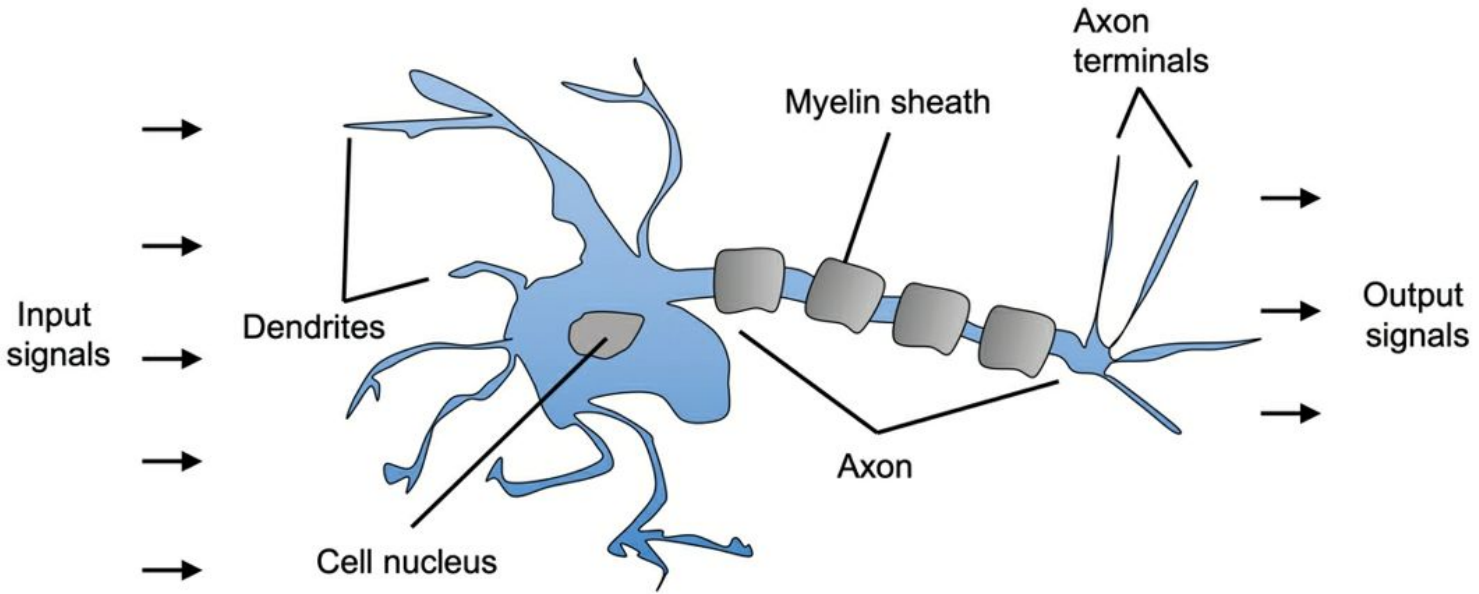
Ref: Raschka, Sebastian; Liu, Yuxi (Hayden); Mirjalili, Vahid. Machine Learning with PyTorch and Scikit-Learn: Develop machine learning and deep learning models with Python

Outline

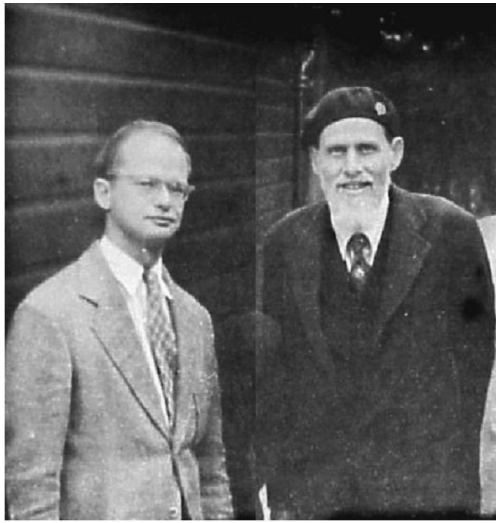
- Artificial neurons and neural networks
- Implementing and training multilayer neural network from scratch

Biological neuron

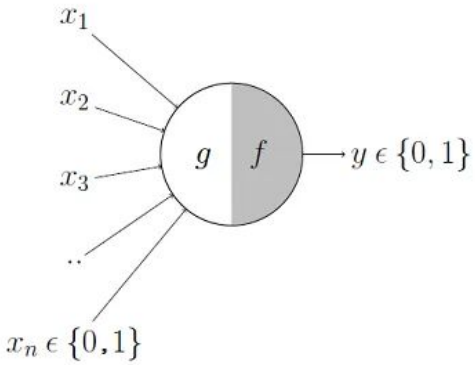
- Interconnected nerve cells in the brain



History of artificial neuron



McCulloch-Pitts (MCP) neuron - 1943 (A Logical Calculus of the Ideas Immanent in Nervous Activity by W. S. McCulloch and W. Pitts, Bulletin of Mathematical Biophysics, 5(4): 115-133, 1943).

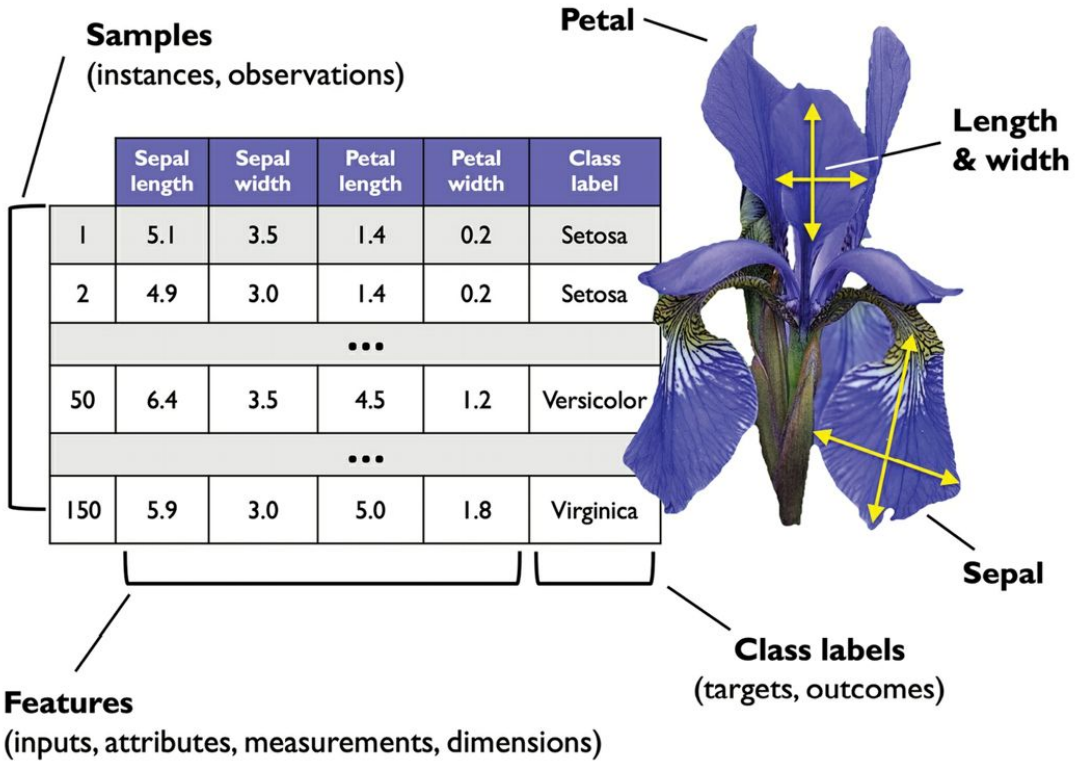


- Frank Rosenblatt published the first concept of the perceptron learning rule based on the MCP neuron model - 1957 (The Perceptron: A Perceiving and Recognizing Automaton by F. Rosenblatt, Cornell Aeronautical Laboratory, 1957).

- Rosenblatt proposed an algorithm that would automatically learn the optimal weight coefficients that would then be multiplied with the input features in order to make the decision of whether a neuron fires (transmits a signal) or not.

Ref: https://www.historyofinformation.com/detail.php?entryid=782

A Sample dataset to explain notations



	Sepal length	Sepal width	Petal length	Petal width	Class label
1	5.1	3.5	1.4	0.2	Setosa
2	4.9	3.0	1.4	0.2	Setosa
...					
50	6.4	3.5	4.5	1.2	Versicolor
...					
150	5.9	3.0	5.0	1.8	Virginica

This dataset has 150 examples.

$$X = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & x_3^{(1)} & x_4^{(1)} \\ x_1^{(2)} & x_2^{(2)} & x_3^{(2)} & x_4^{(2)} \\ \vdots & \vdots & \vdots & \vdots \\ x_1^{(150)} & x_2^{(150)} & x_3^{(150)} & x_4^{(150)} \end{bmatrix}$$

Explaining notations

A row of X→ one flower instance

$$\boldsymbol{X}^{(i)} = \begin{bmatrix} x_1^{(i)} & x_2^{(i)} & x_3^{(i)} & x_4^{(i)} \end{bmatrix}$$

Feature dimension is 150 (for this dataset), which is a column

$$\boldsymbol{x_j} = \begin{bmatrix} x_j^{(1)} \\ x_j^{(2)} \\ \cdots \\ x_j^{(150)} \end{bmatrix}$$

Target variables:

$$\boldsymbol{y} = \begin{bmatrix} y^{(1)} \\ \cdots \\ y^{(150)} \end{bmatrix}, \text{ where } y^{(i)} \in \{\text{Setosa, Versicolor, Virginica}\}$$

The formal definition of an artificial neuron

$$\boldsymbol{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}, \quad \boldsymbol{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix}$$

The corresponding weight vector (w)

Input values (x)

Net input

$$z = w_1 x_1 + w_2 x_2 + \ldots + w_m x_m$$

Unit step function

$$\sigma(z) = \begin{cases} 1 & \text{if } z \geq \theta \\ 0 & \text{otherwise} \end{cases}$$

$$z \geq \theta$$

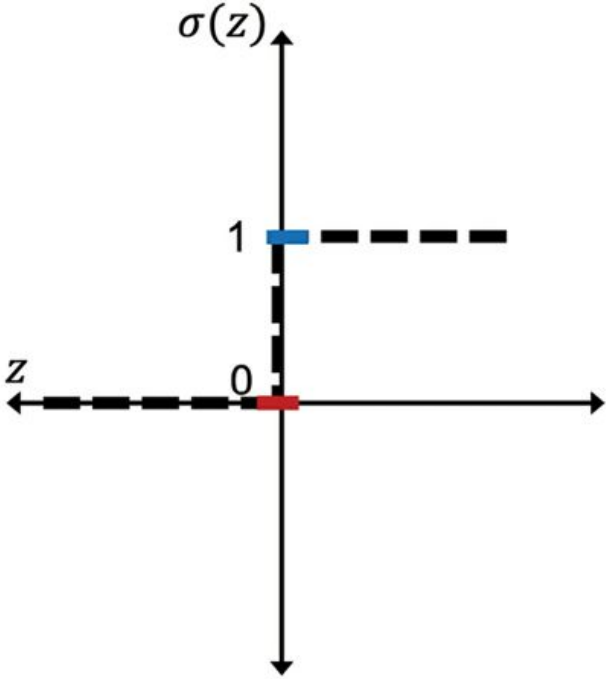
$$z - \theta \geq 0$$

$$z = w_1 x_1 + \ldots + w_2 x_2 + b = w^T x + b$$

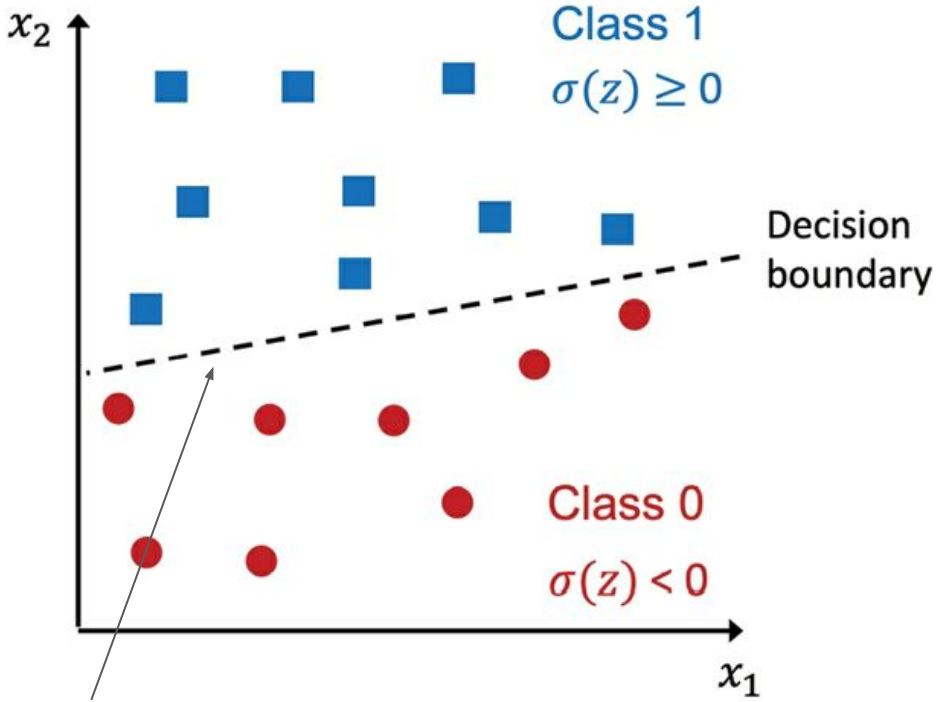
$$-\theta = b$$

$$\sigma(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

Decision function of the perceptron

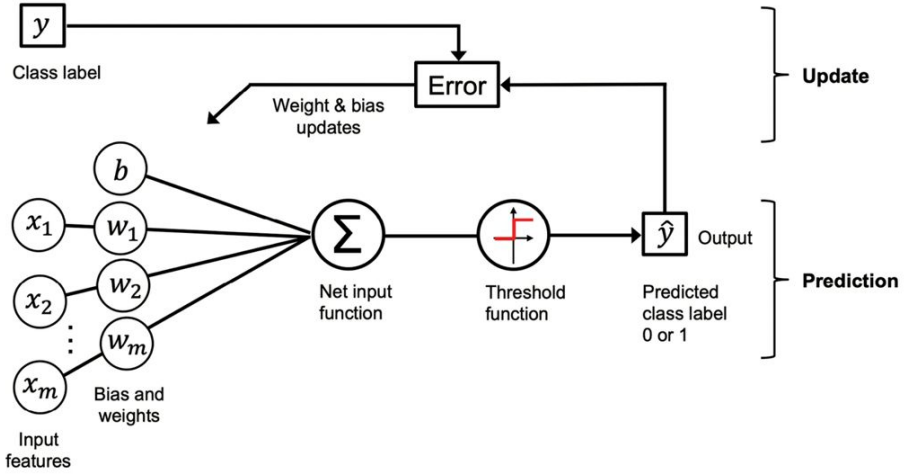


where $z = \boldsymbol{w}^T\boldsymbol{x} + b$



Linear decision boundary

The perceptron learning rule



1. Initialize the **w** and **b** unit to 0 or small random numbers
2. For each training sample
   a. Computer output (predictions - y^)
   b. Update the **w** and **b**

$$w_j := w_j + \Delta w_j$$

$$\text{and} \quad b := b + \Delta b$$

$$\Delta w_j = \eta\left(y^{(i)} - \hat{y}^{(i)}\right)x_j^{(i)}$$

$$\text{and} \quad \Delta b = \eta\left(y^{(i)} - \hat{y}^{(i)}\right)$$

Note that, no x here

Example (1/2)

If two input features are there, we update all weights and bias unit simultaneously.

$$\Delta w_1 = \eta(y^{(i)} - \text{output}^{(i)})x_1^{(i)};$$
$$\Delta w_2 = \eta(y^{(i)} - \text{output}^{(i)})x_2^{(i)};$$
$$\Delta b = \eta(y^{(i)} - \text{output}^{(i)})$$

What happens if predictions are 100% correct?

$$(1)\ y^{(i)} = 0, \quad \hat{y}^{(i)} = 0, \quad \Delta w_j = \eta(0-0)x_j^{(i)} = 0, \quad \Delta b = \eta(0-0) = 0$$

$$(2)\ y^{(i)} = 1, \quad \hat{y}^{(i)} = 1, \quad \Delta w_j = \eta(1-1)x_j^{(i)} = 0, \quad \Delta b = \eta(1-1) = 0$$

Example (2/2)

In the case of wrong predictions,

$$(3)\ y^{(i)} = 1,\quad \hat{y}^{(i)} = 0,\quad \Delta w_j = \eta(1-0)x_j^{(i)} = \eta x_j^{(i)},\quad \Delta b = \eta(1-0) = \eta$$

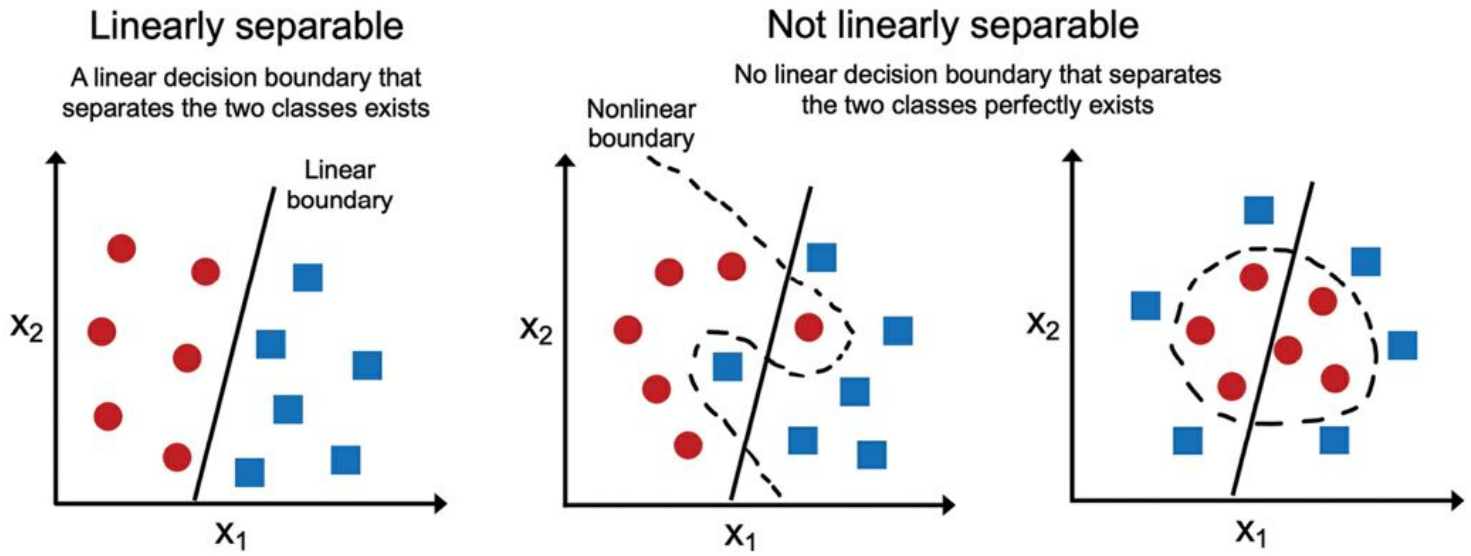
$$(4)\ y^{(i)} = 0,\quad \hat{y}^{(i)} = 1,\quad \Delta w_j = \eta(0-1)x_j^{(i)} = -\eta x_j^{(i)},\quad \Delta b = \eta(0-1) = -\eta$$

Compare when x_j =1.5 and 2

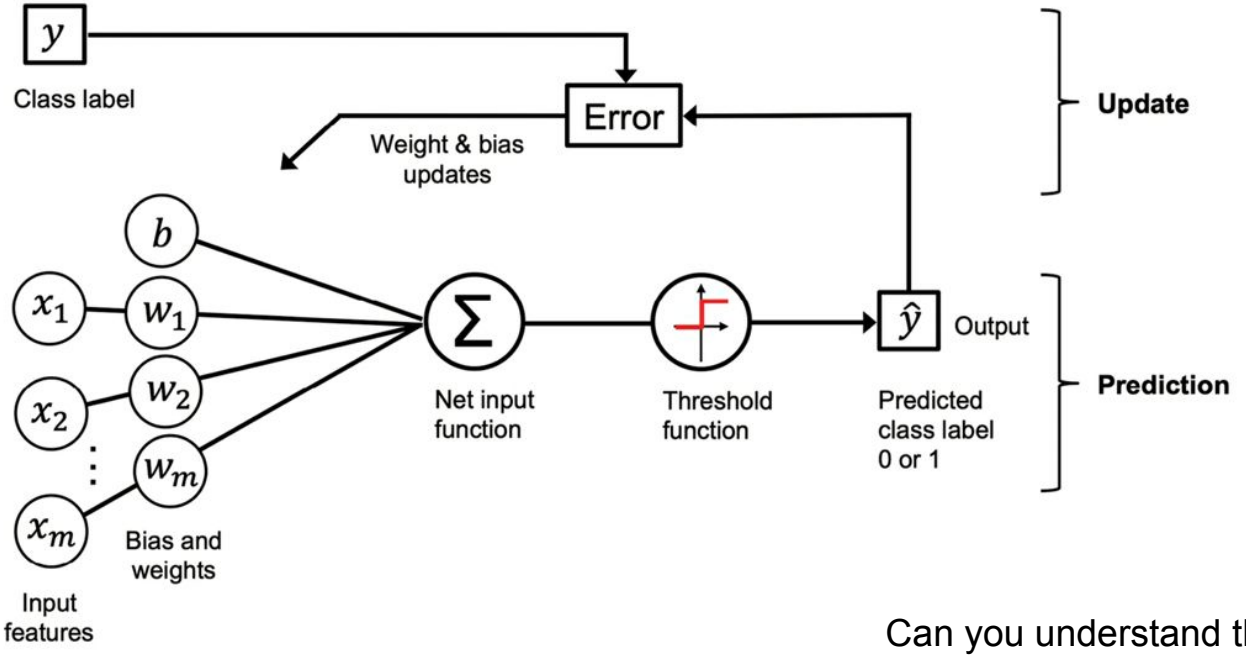
$$\Delta w_j = (1-0)1.5 = 1.5,\quad \Delta b = (1-0) = 1$$

$$\Delta w_j = (1-0)2 = 2,\quad \Delta b = (1-0) = 1$$

Note: convergence of the perceptron - if linearly separable



The big picture

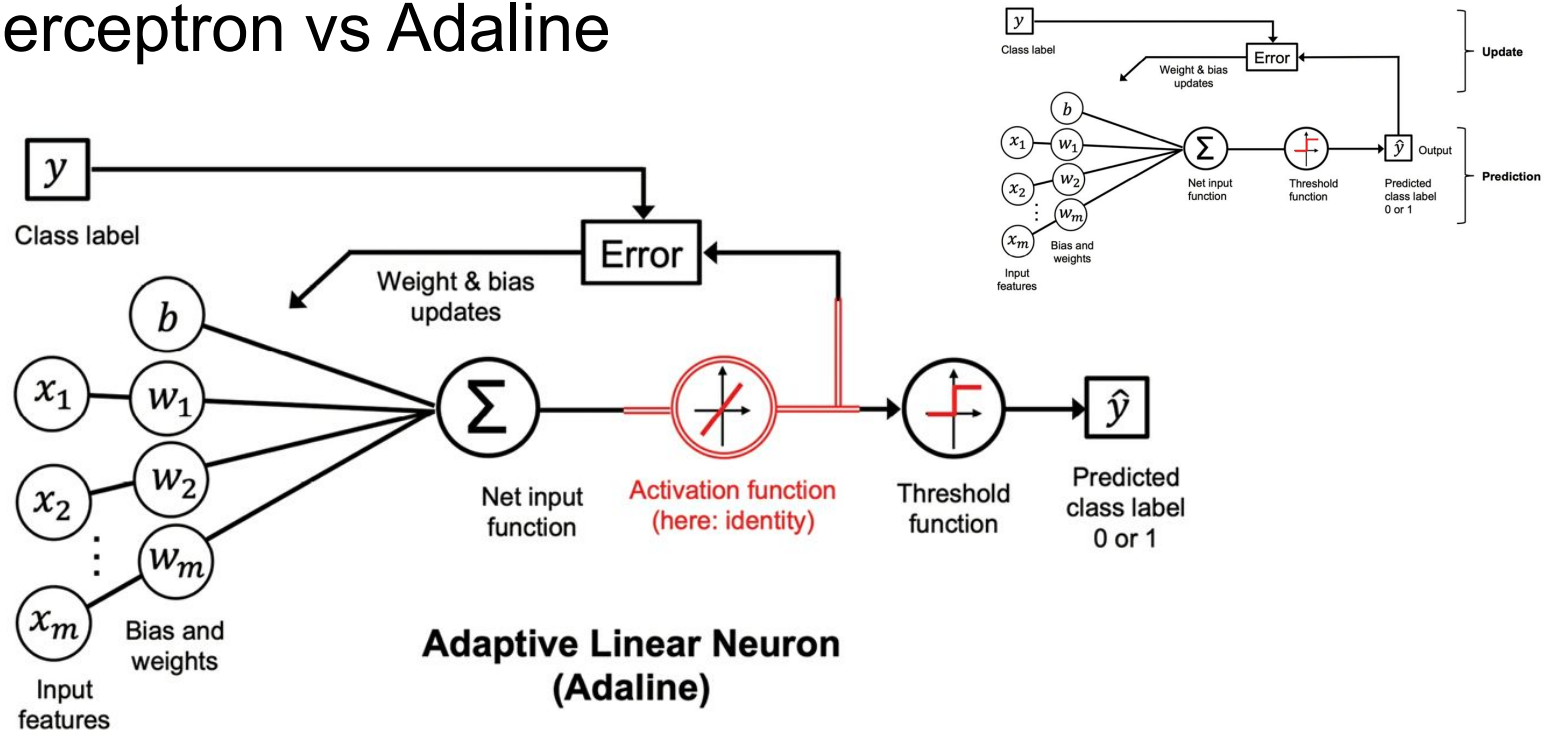


Can you understand this now?

ADAptive LInear NEuron (ADALINE) - (Widrow-Hoff rule)

- Adaline was published by Bernard Widrow and his doctoral student Tedd Hoff only a few years after Rosenblatt's perceptron algorithm.
- The Adaline algorithm is particularly interesting because it illustrates the key concepts of defining and minimizing continuous loss functions.
- The weights are updated based on a **linear activation function** rather than a unit step function.

Perceptron vs Adaline



The Adaline algorithm compares the true class labels with the linear activation function's continuous valued output to compute the model error and update the weights. In contrast, the perceptron compares the true class labels to the predicted class labels.

Minimizing loss function (objective function) - gradient descent

Sample loss function:

Mean squared error (MSE)

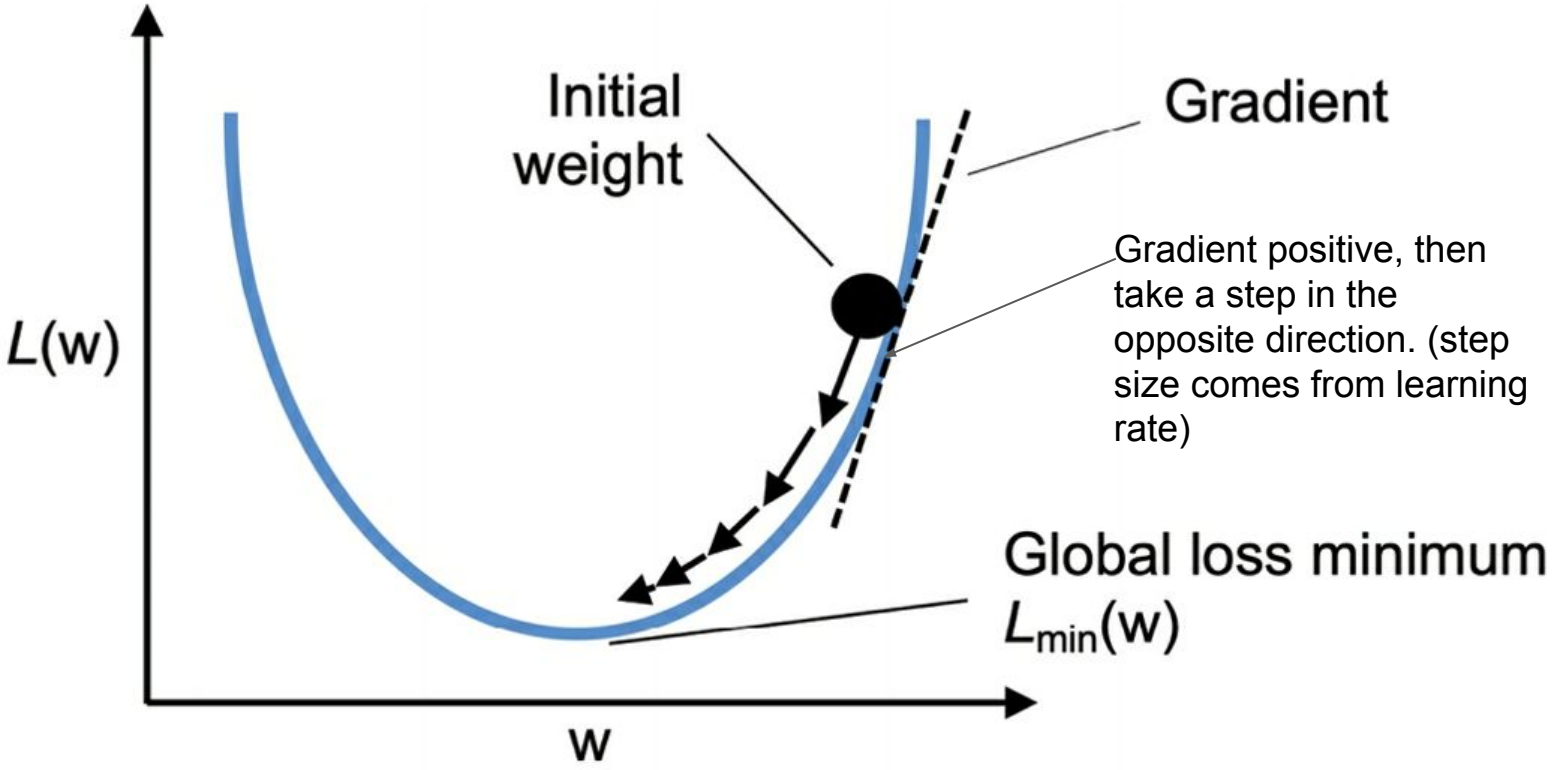
$$L(\boldsymbol{w}, b) = \frac{1}{2n}\sum_{i=1}^{n}\left(y^{(i)} - \sigma\left(z^{(i)}\right)\right)^2$$

Just for authors' convenience ;-)

Because of linear activation functions,

- The loss function becomes differentiable
- This loss function is convex
- Then we can use **gradient descent optimization** algorithm

How gradient descent works?



How gradient descent works?

$$\boldsymbol{w} := \boldsymbol{w} + \Delta\boldsymbol{w}, \quad b := b + \Delta b$$

$$\Delta\boldsymbol{w} = -\eta\nabla_w L(\boldsymbol{w}, b), \quad \Delta b = -\eta\nabla_b L(\boldsymbol{w}, b)$$

Gradient of the loss function L(w,b)

How to calculate?

$$\frac{\partial L}{\partial w_j} = -\frac{2}{n}\sum_i \left(y^{(i)} - \sigma\left(z^{(i)}\right)\right) x_j^{(i)}$$

$$\frac{\partial L}{\partial b} = -\frac{2}{n}\sum_i \left(y^{(i)} - \sigma\left(z^{(i)}\right)\right)$$

$$\Delta w_j = -\eta\frac{\partial L}{\partial w_j} \text{ and } \Delta b = -\eta\frac{\partial L}{\partial b}$$

How do we calculate MSE derivative

$$\frac{\partial L}{\partial w_j} = \frac{\partial}{\partial w_j}\frac{1}{n}\sum_i \left(y^{(i)} - \sigma\left(z^{(i)}\right)\right)^2 = \frac{1}{n}\frac{\partial}{\partial w_j}\sum_i \left(y^{(i)} - \sigma\left(z^{(i)}\right)\right)^2$$

$$= \frac{2}{n}\sum_i \left(y^{(i)} - \sigma\left(z^{(i)}\right)\right)\frac{\partial}{\partial w_j}\left(y^{(i)} - \sigma\left(z^{(i)}\right)\right)$$

$$= \frac{2}{n}\sum_i \left(y^{(i)} - \sigma\left(z^{(i)}\right)\right)\frac{\partial}{\partial w_j}\left(y^{(i)} - \sum_i \left(w_j x_j^{(i)} + b\right)\right)$$

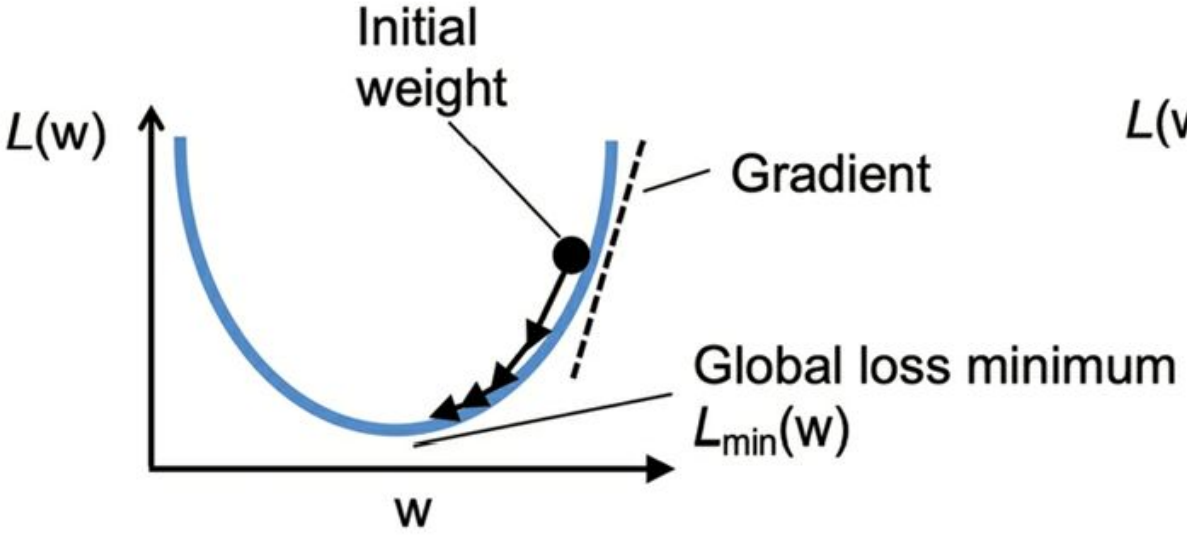
$$= \frac{2}{n}\sum_i \left(y^{(i)} - \sigma\left(z^{(i)}\right)\right)\left(-x_j^{(i)}\right) = -\frac{2}{n}\sum_i \left(y^{(i)} - \sigma\left(z^{(i)}\right)\right)x_j^{(i)}$$

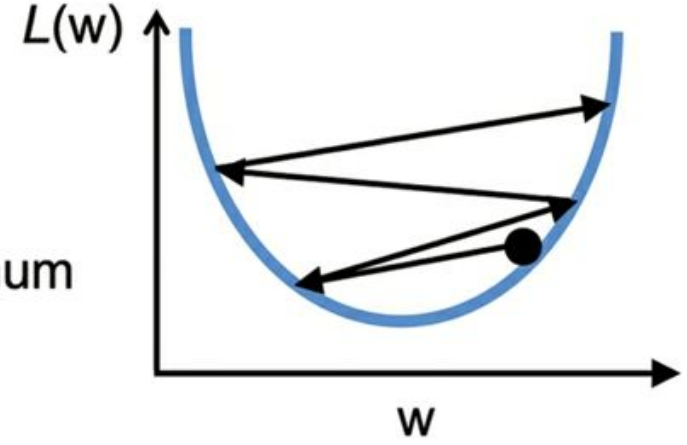
The same approach can be used to find partial derivative $\frac{\partial L}{\partial b}$ except that $\frac{\partial}{\partial b}\left(y^{(i)} - \sum_i \left(w_j^{(i)} x_j^{(i)} + b\right)\right)$ is equal to –1 and thus the last step simplifies to $-\frac{2}{n}\sum_i \left(y^{(i)} - \sigma\left(z^{(i)}\right)\right)$.

Why Adaline

- Z(.) output is a real number, not an integer class
- Weight update is calculated using all examples (full batch gradient descent)

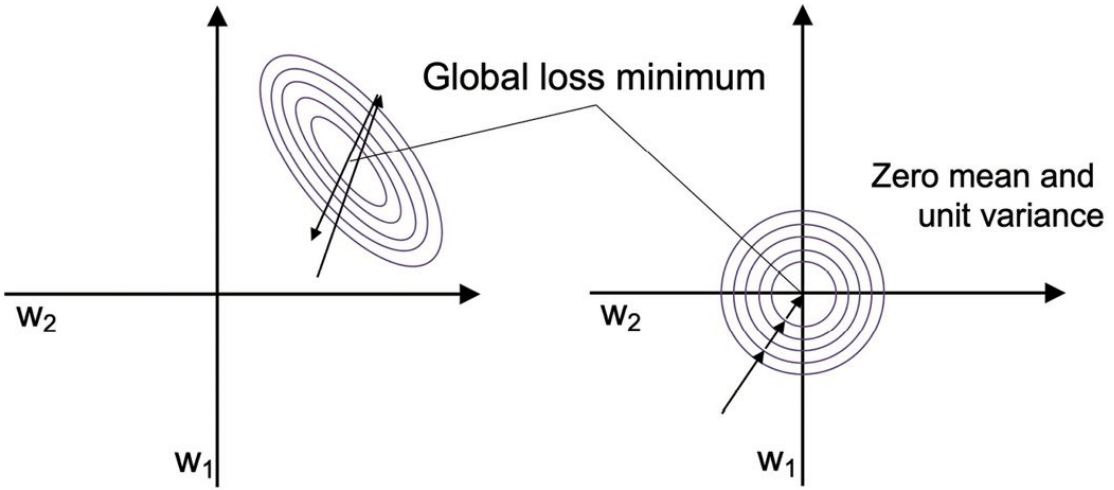
Why do we need a good learning rate?





Feature scaling - standardization

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}$$



Large scale ML and stochastic gradient descent

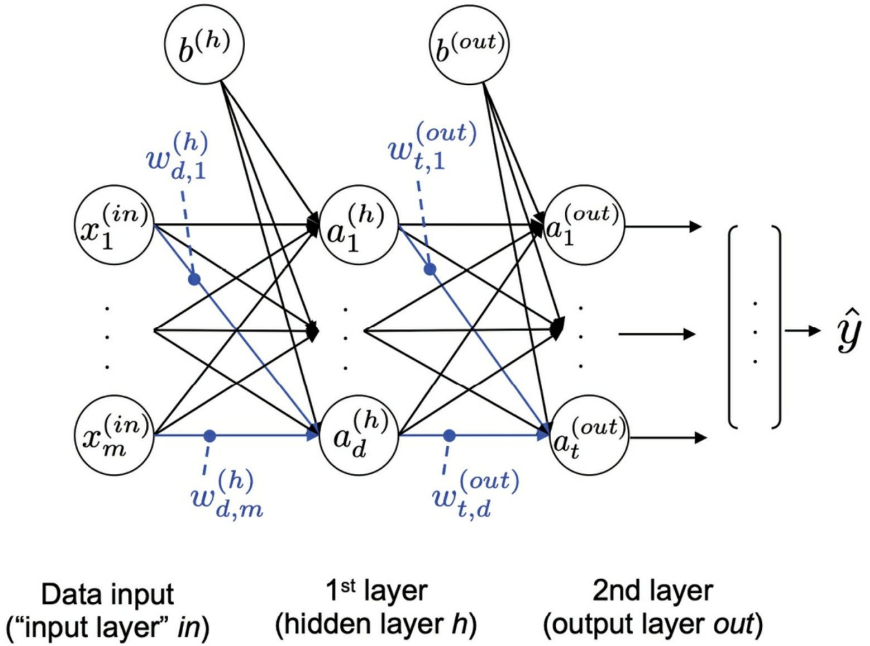
- We know full batch gradient descent, but what is the problem with a large dataset?
- One alternative → **stochastic gradient descent (SGD)**
- Other names for SGD → iterative or online gradient descent, mini-batch gradient descent
- SGD can be used for **online learning**
- In SGD → we normally use adaptive learning rate, for ex,

$$\frac{c_1}{[\text{number of iterations}] + c_2}$$

Implementing a multilayer ANN from scratch

Multilayer Perceptron (MLP)

Connecting single neurons to multilayer feedforward neural network.



If a network has more than one hidden layer, then it is called **Deep NN.**

Special algorithms to train DNNs → **Deep Learning**

To know: One-hot representation

One-hot representation for three classes

$$0 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, 1 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, 2 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

To be continued…!

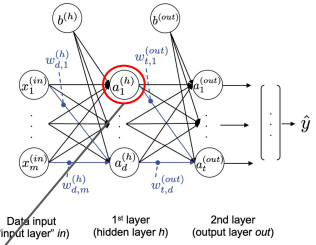
MLP learning procedure

Forward propagate the patterns of the training data

Calculate loss

Backpropagate the loss

Find its derivative with respect to each weight and bias unit in the network, and update the model.

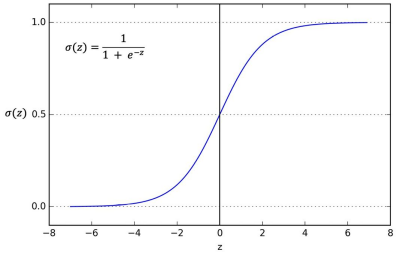


$$z_1^{(h)} = x_1^{(in)} w_{1,1}^{(h)} + x_2^{(in)} w_{1,2}^{(h)} + \cdots + x_m^{(in)} w_{1,m}^{(h)}$$

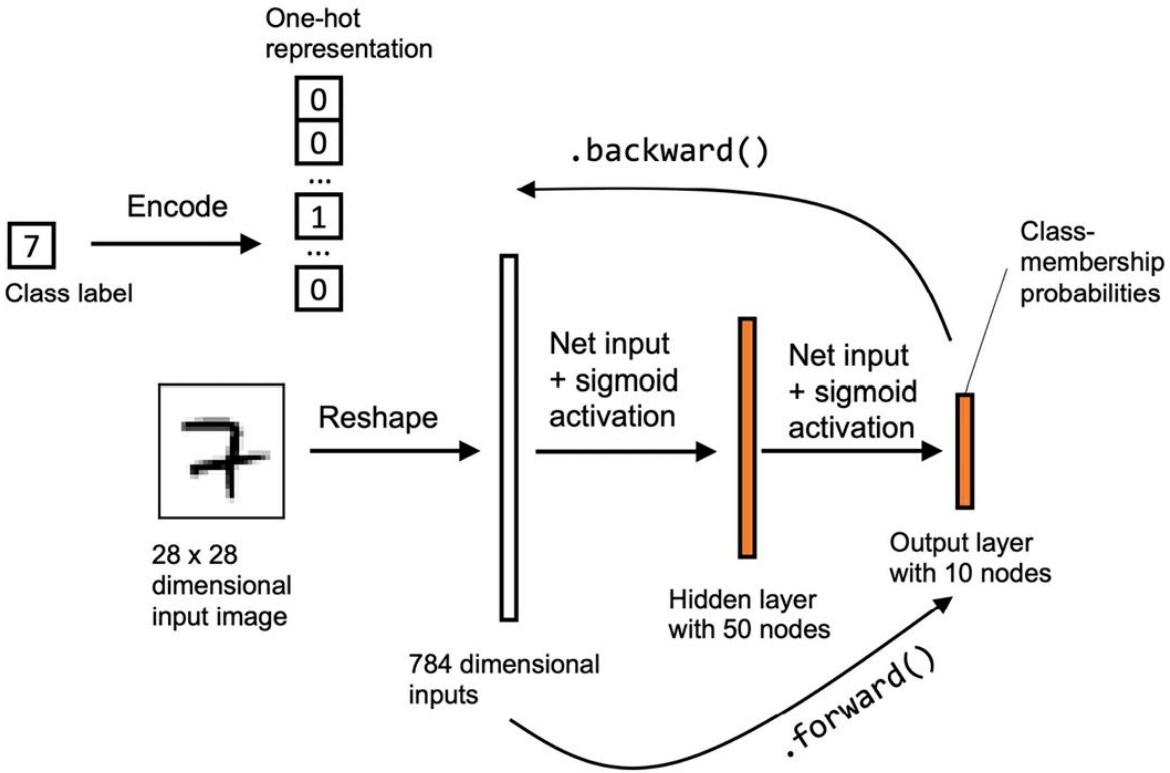
$$a_1^{(h)} = \sigma\left(z_1^{(h)}\right)$$

We need nonlinear activation functions

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

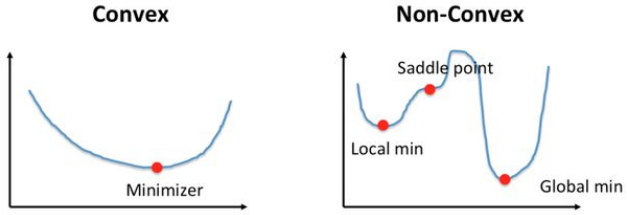


Ex: Labeling handwritten digits



Backpropagation

- Why backpropagation? To compute the partial derivatives of a complex, non-convex function



Activation functions

Activation function	Equation	Example	1D graph
Linear	$\sigma(z) = z$	Adaline, linear regression	
Unit step (Heaviside function)	$\sigma(z) = \begin{cases} 0 & z < 0 \\ 0.5 & z = 0 \\ 1 & z > 0 \end{cases}$	Perceptron variant	
Sign (signum)	$\sigma(z) = \begin{cases} -1 & z < 0 \\ 0 & z = 0 \\ 1 & z > 0 \end{cases}$	Perceptron variant	
Piece-wise linear	$\sigma(z) = \begin{cases} 0 & z \leq -½ \\ z + ½ & -½ \leq z \leq ½ \\ 1 & z \geq ½ \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\sigma(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, multilayer NN	
Hyperbolic tangent (tanh)	$\sigma(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$	Multilayer NN, RNNs	
ReLU	$\sigma(z) = \begin{cases} 0 & z < 0 \\ z & z > 0 \end{cases}$	Multilayer NN, CNNs	

The End

Single-layer neural network